Softablitz 2026

Team: The Three Musketeers

Team ID : 220

Team Info

Member	Name	Reg No.	Branch	Discord Username
Member 1	Naman Kumar	20253167	CSE	krnaman2007
Member 2	Om Upadhyay	20253181	CSE	upadhyayom1_37905
Member 3	Adya Tripathi	20253026	CSE	adya_tripathi

App name : AuraDesk

Description of the problem

Traditional desktop wallpaper customization requires users to search multiple platforms, download images manually, and repeatedly change desktop settings. Existing solutions often lack unified collection management and intelligent automation, limiting personalization and convenience.

AuraDesk is a unified desktop application designed to simplify wallpaper discovery, organization, and customization. It enables users to explore wallpapers from external image sources, preview images with descriptions and source credits, and apply them directly to their desktop with appropriate scaling and cropping.

Users can create named wallpaper collections, combine online images with locally stored wallpapers, and configure automatic shuffling or rotation at preferred intervals. The application further introduces context-aware personalization by adapting wallpapers according to local time, weather conditions, and calendar events. With persistent rotation history, manual override controls, and system-tray operation, The Reality Weaver delivers a convenient, organized, and dynamic desktop personalization experience with minimal manual intervention.

Process flow of the app

1. Wallpaper Discovery & Search: Users browse, search, and filter wallpapers from external image sources, with each result displaying its preview, description, and source credits.

2. Wallpaper Preview & Application: Users preview their selected wallpaper before applying it directly to the desktop, with automatic scaling and cropping for the screen.

3. Collection Management: Users create named collections and organize wallpapers from online sources or local storage for easy reuse and personalization.

4. Wallpaper Rotation & Scheduling: Users configure shuffling or automatic rotation at preferred intervals, while the application maintains rotation history and avoids recently displayed wallpapers.

5. Context-Aware Personalization: The application selects wallpapers based on configured local-time and weather rules, allowing users to enable or disable automation and manually override selections.

6. Calendar Integration & Persistent Automation: Calendar events trigger associated wallpaper collections, while scheduled operations continue through the system tray and rotation history persists across application restarts.

FEATURES ANNOUNCEMENT

1. Wallpaper Discovery & Search: Search, browse, and filter wallpapers from external image sources, with previews, descriptions, and source credits.

2. Direct Wallpaper Application: Apply wallpapers directly to the desktop with automatic scaling and cropping, eliminating manual downloads and repeated system-setting changes.

3. Wallpaper Collections: Create named collections, save wallpapers from online sources or local storage, and organize preferred images for convenient access.

4. Automated Wallpaper Rotation: Shuffle or cycle through selected collections at configurable intervals, preserve rotation history across restarts, and avoid recently displayed wallpapers.

5. Context-Aware Personalization: Adapt wallpapers dynamically to local time and current weather through configurable rules, with options to disable automation or manually override selections.

6. Calendar-Based Wallpaper Scheduling: Integrate calendar events with personalized wallpaper collections to schedule wallpaper changes around meetings and other events.

7. Persistent Background Operation: Continue scheduled wallpaper automation through the system tray when the application is minimized, maintaining rotation history across application restarts.

External libraries/APIs

Library/API	Purpose	Why chosen over alternatives ?
Electron	Desktop application shell: window management, system tray, and background execution when minimized.	Gives native access to the desktop, tray, and file system using web technologies, with one codebase across Windows, macOS and Linux.
React	Frontend UI: wallpaper search and filters,preview screen, collection manager, and settings panel.	Component-based design lets result grids, previews and settings update quickly without manual DOM handling.
Node.js	Application logic: calling external APIs, rotation workflow, personalization rules, and file handling.	Asynchronous, event-driven runtime handles network calls and background timers without blocking the UI. Ships with Electron.
wallpaper(npm)	Setting the desktop wallpaper directly from the application.	Cross-platform package that applies an image to the desktop without manual system-setting changes.
Sharp	Image scaling and cropping to fit the screen resolution and aspect ratio.	Much faster than pure-JavaScript image libraries and handles resize and crop with low memory use.
SQLite(better-sqlite3)	Local persistence: collections, collection items,rotation history, rules, and app settings.	Serverless, file-based database that needs no setup and keeps history intact across application restarts.
Axios	HTTP requests to wallpaper sources, weather service, and calendar service.	Simple promise-based API with built-in error handling, timeouts and JSON parsing.
node-cron	Rotation scheduling at user-defined intervals and time-based personalization rules.	Lightweight in-process scheduler; no external job service is required for a desktop app.
Unsplash API / Pexels API	Wallpaper discovery: search, filters, image previews, descriptions and photographer credits.	Free, well-documented APIs with high-resolution images and credit information included in every response.
OpenWeatherMap API	Current weather condition and location data for weather-based wallpaper rules.	Free tier with simple REST access to current conditions (clear, rain, cloudy, etc.).
Google Calendar API	Fetching upcoming events to link them with wallpaper collections.	Widely used calendar provider with official OAuth support and event start/end time data.

Core Engineering Architecture

AuraDesk uses a modular, event-driven architecture built with Electron, React, Node.js, SQLite, and external APIs. It separates wallpaper discovery, management, scheduling, desktop integration, and personalization for reliable operation.

Infrastructure Components

1. Desktop Layer: Electron and React provide the cross-platform interface, while the main process handles desktop operations, file access, and system-tray integration through secure IPC.

2. Application Services: Node.js services manage wallpaper discovery, collections, rotation, and context-aware personalization independently of the UI.

3. External APIs: Axios connects to Unsplash/Pexels, OpenWeatherMap, and Google Calendar. Timeouts, validation, retries, and OAuth support reliable and secure requests.

4. Image Processing: Wallpaper and Sharp packages apply wallpapers with appropriate resizing and cropping while limiting memory usage.

5. Local Storage: SQLite stores settings, collections, personalization rules, and rotation history. Transactions maintain consistency and restore state after restarts.

6. Scheduling & Lifecycle: node-cron manages scheduled rotations, while the system tray supports background operation, manual rotation, and automation controls.

Application Data Flow

1. Wallpaper Discovery: Users search or filter wallpapers; the discovery service fetches and normalizes image metadata and attribution through external APIs.

2. Preview & Application: Selected images are validated, resized or cropped, and applied to the desktop. History updates only after successful application.

3. Collection Management: Collection changes and settings are validated and saved to SQLite, then restored when the application restarts.

4. Scheduled Rotation: The scheduler selects wallpapers based on configured intervals and shuffle preferences, avoiding recently displayed images where possible.

5. Context-Aware Personalization: The engine evaluates time, weather, and calendar events to select suitable wallpapers. Manual overrides take priority over automatic rules.

6. Background Execution & Recovery: The system tray keeps scheduling active when the window is hidden. On restart, settings and history are restored; unavailable APIs trigger fallback to local collections while retaining the current wallpaper.

Engineering Challenge	Naive/Basic Approach	Our Smart Engineering Approach	Why It Wins
Desktop Security	Allow the renderer to access Node.js and operating-system APIs directly.	Use Electron context isolation, disabled renderer Node integration, and validated IPC handlers.	Reduces the impact of compromised web content and restricts privileged operations.
API Reliability	Make unlimited requests and wait indefinitely for external services.	Apply request timeouts, bounded retries, response validation, and controlled concurrency.	Prevents stalled operations and improves responsiveness during network failures.
Image Processing	Load full-resolution images into memory and resize them using unbounded operations.	Use Sharp, bounded processing, and temporary files with cleanup.	Reduces memory pressure and improves image preparation efficiency.
Persistent State	Store collections, settings, and history independently in unrelated files.	Use SQLite transactions and structured records for settings, collections, and rotation history.	Preserves consistency and allows reliable recovery after application restarts.
Rotation Scheduling	Use independent timers that may overlap or restart unpredictably.	Maintain a single coordinated scheduler with persisted configuration and guarded rotation operations.	Avoids duplicate rotations and supports predictable scheduling.
Context-Aware Selection	Fetch weather and calendar data on every rotation without checking freshness.	Reuse valid cached context, apply configurable rules, and fall back to local collections when integrations fail.	Reduces unnecessary requests and keeps personalization functional during outages.
Cross-Platform Integration	Assume identical wallpaper-setting behavior on every operating system.	Isolate wallpaper application behind WallpaperManager and validate behavior on supported operating systems.	Simplifies platform-specific handling and makes failures easier to diagnose.
Application Recovery	Assume scheduled operations and the last wallpaper state survive automatically.	Persist settings and history, restore state on startup, and reinitialize scheduling explicitly.	Improves continuity after crashes, restarts, and temporary service failures.
Performance Validation	Assume the application is responsive because it works on one device.	Test API failures, large images, repeated rotations, restart recovery, and resource consumption under concurrent operations.	Identifies bottlenecks and reliability issues before release.

CLASS DIAGRAM

